



BLM4538 - IOS ile Mobil Uygulama Geliştirme

HABİBE ALEYNA TAŞDEMİR 22290583

1. Proje Tanımı ve Amaç

1.1 Projenin Tanımı

Bu proje, kozmetik ürünlerin içeriklerini analiz ederek kullanıcıya içerik güvenliği hakkında bilgi veren bir mobil uygulamanın geliştirilmesini amaçlamaktadır. Uygulama sayesinde kullanıcılar kozmetik ürünleri arayabilir, barkod veya kamera ile ürünleri tarayabilir ve ürün içeriklerinin güvenlik değerlendirmelerini görebilir.

Uygulama mobil platformlarda çalışacak olup React Native kullanılarak geliştirilecek ve backend tarafında .NET tabanlı bir REST API kullanılacaktır.

1.2 Projenin Amacı

Projenin temel amacı, kozmetik ürün kullanıcılarının ürün içeriklerini kolay bir şekilde analiz edebilmesini sağlamaktır. Birçok kozmetik ürün içeriği teknik terimler içerdiğinden kullanıcılar bu içeriklerin güvenli olup olmadığını değerlendirmekte zorlanmaktadır.

Geliştirilecek sistem, ürün içeriklerini analiz ederek kullanıcıya şu bilgileri sunacaktır:

- içerik güvenlik durumu
- içerik açıklamaları
- ürün güvenlik skoru
- potansiyel riskli içerikler

1.3 Uygulamanın Temel Özellikleri

Uygulama aşağıdaki temel özellikleri içerecektir:

- ürün arama sistemi
- barkod ile ürün tarama
- kamera ile ürün adı tanıma (OCR)
- ürün içeriklerinin listelenmesi
- içerik güvenlik analizi
- ürün güvenlik skorunun hesaplanması
- favori ürün listesi
- kullanıcı tarama geçmişi
- kullanıcı cilt profili

1.4 Kullanıcı Senaryosu

Bir kullanıcı uygulamayı açtığında aşağıdaki işlemleri gerçekleştirebilir:

1. ürün adı ile arama yapabilir
2. barkod taratarak ürün bulabilir
3. kamera ile ürün adı algılayabilir
4. ürün içerik listesini görüntüleyebilir

5. içeriklerin güvenlik durumunu inceleyebilir
6. ürünü favorilere ekleyebilir
7. daha önce yaptığı taramaları görüntüleyebilir

Bu süreç kullanıcıya hızlı ve anlaşılır bir içerik analizi sunmayı amaçlar.

1.5 Projenin Teknik Yapısı

Proje üç ana bileşenden oluşacaktır:

1. mobil uygulama (React Native)
2. backend servisleri (.NET Web API)
3. veritabanı sistemi (PostgreSQL veya MongoDB)

Mobil uygulama kullanıcı arayüzünü sağlayacak, backend sistemi veri işleme ve analiz işlemlerini gerçekleştirecek, veritabanı ise ürün ve içerik verilerini saklayacaktır.

1.6 Projenin Genel İşleyişi

Sistemin genel çalışma prensibi aşağıdaki gibidir:

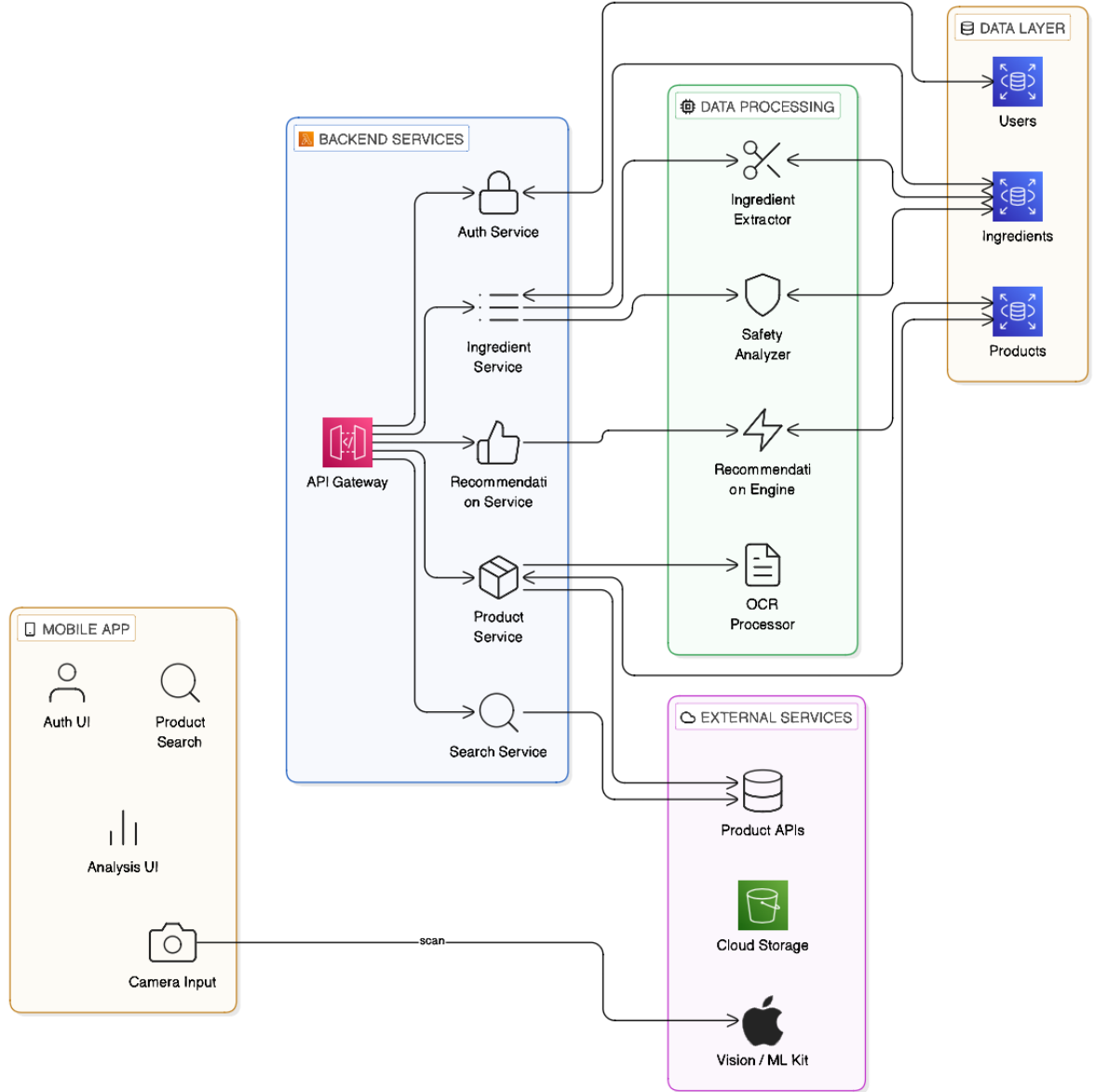
1. kullanıcı ürün araması yapar veya ürünü tarar
2. sistem ürünü veritabanında arar
3. ürün bulunursa içerik listesi alınır
4. içerikler güvenlik veritabanı ile eşleştirilir
5. ürün güvenlik skoru hesaplanır
6. sonuç kullanıcıya görsel bir analiz ekranında sunulur

2. Sistem Mimarisi

2.1 Genel Sistem Mimarisi

Uygulama üç ana bileşenden oluşan bir istemci–sunucu mimarisi kullanacaktır. Bu mimari yapı mobil uygulama, backend servisleri ve veritabanı katmanlarından oluşmaktadır.

Genel mimari yapı aşağıdaki gibidir:



Mobil uygulama kullanıcı arayüzünü sağlar ve kullanıcı etkileşimlerini yönetir. Backend sistemi ise veri işleme, analiz ve veri erişim işlemlerini gerçekleştirir.

2.2 Katmanlı Mimari Yapı

Backend sistemi katmanlı bir mimari yapıya sahip olacaktır. Bu mimari sistemin daha modüler ve sürdürülebilir olmasını sağlar.

Backend katmanları şu şekildedir:

1. **Controller Layer**
2. **Service Layer**
3. **Repository Layer**
4. **Database Layer**

Bu yapı sayesinde veri erişimi, iş mantığı ve API kontrolü birbirinden ayrılmış olur.

2.3 Frontend – Backend İletişimi

Mobil uygulama ile backend sistemi arasındaki iletişim **REST API** üzerinden gerçekleşecektir.

Mobil uygulama aşağıdaki işlemler için API çağrıları yapacaktır:

- ürün arama
- ürün detaylarını getirme
- barkod ile ürün bulma
- ingredient analiz sonuçlarını getirme
- favori ürün işlemleri
- kullanıcı profil bilgileri

Tüm veri alışverişi **JSON formatında** yapılacaktır.

2.4 Veri Akışı (Data Flow)

Sistemin veri akışı aşağıdaki şekilde gerçekleşir:

1. kullanıcı mobil uygulama üzerinden bir işlem başlatır
2. mobil uygulama backend API'ye bir istek gönderir
3. backend sistemi isteği işler
4. gerekli veri veritabanından alınır
5. analiz işlemleri yapılır
6. sonuç JSON formatında mobil uygulamaya gönderilir

Bu veri akışı sistemin hızlı ve güvenilir çalışmasını sağlar.

2.5 Ürün Arama Akışı

Ürün arama işlemi aşağıdaki adımlarla gerçekleşir:

1. kullanıcı arama ekranına ürün adı girer

2. mobil uygulama backend'e bir arama isteđi gönderir
3. backend sistemi ürün veritabanında arama yapar
4. bulunan ürünler mobil uygulamaya gönderilir
5. kullanıcı ürün detay ekranını açabilir

Arama işlemleri veritabanı indeksleri kullanılarak optimize edilecektir.

2.6 Barkod Tarama Akışı

Barkod ile ürün bulma işlemi aşağıdaki adımlarla gerçekleşir:

1. kullanıcı tarama ekranını açar
2. kamera barkodu algılar
3. barkod numarası alınır
4. barkod backend API'ye gönderilir
5. backend veritabanında barkod eşleşmesi yapar
6. ürün bulunursa ürün bilgileri mobil uygulamaya gönderilir

Barkod tanıma işlemi cihaz üzerinde gerçekleşecektir.

2.7 OCR (Metin Tanıma) Akışı

Barkod bulunamadığı durumlarda ürün adı kamera ile okunabilir.

OCR süreci şu adımlarla gerçekleşir:

1. kullanıcı ürünün fotoğrafını çeker
2. görüntü Apple Vision OCR sistemi ile işlenir
3. metin verisi elde edilir
4. metin temizlenir ve normalize edilir
5. backend sisteminde ürün araması yapılır
6. en yakın eşleşen ürün kullanıcıya gösterilir

OCR işlemi cihaz üzerinde gerçekleştirilecektir.

2.8 Ingredient Analiz Süreci

Bir ürün bulunduğunda içerik analizi yapılacaktır.

Analiz süreci şu şekilde gerçekleşir:

1. ürün veritabanından alınır
2. ürünün ingredient listesi elde edilir
3. ingredient listesi ingredient güvenlik veritabanı ile eşleştirilir
4. her içerik için risk seviyesi belirlenir
5. ürün için genel güvenlik skoru hesaplanır
6. sonuç mobil uygulamaya gönderilir

Bu analiz backend sistemi tarafından gerçekleştirilecektir.

3. Mobil Uygulama Tasarımı (React Native)

3.1 Mobil Uygulama Geliştirme Teknolojisi

Mobil uygulama geliştirme sürecinde **React Native** frameworkü kullanılacaktır. React Native, JavaScript ve TypeScript kullanılarak hem iOS hem de Android platformları için tek bir kod tabanı üzerinden mobil uygulama geliştirilmesini sağlayan bir frameworktür. Bu yaklaşım geliştirme sürecini hızlandırırken uygulamanın bakımını ve güncellenmesini de kolaylaştırmaktadır.

Uygulama geliştirme sürecinde **TypeScript** kullanılarak daha güvenli ve sürdürülebilir bir kod yapısı oluşturulacaktır. TypeScript sayesinde statik tip kontrolü sağlanarak geliştirme sürecinde oluşabilecek hatalar erken aşamada tespit edilebilir.

3.2 Navigasyon Yapısı

Uygulama içerisinde sayfalar arası geçişlerin yönetilmesi için **React Navigation** kütüphanesi kullanılacaktır. React Navigation, React Native uygulamalarında en yaygın kullanılan navigasyon çözümlerinden biridir ve uygulama içerisindeki ekranların düzenli bir şekilde yönetilmesini sağlar.

Navigasyon sistemi temel olarak iki yapıdan oluşacaktır:

- **Stack Navigation:** ekranlar arası geçişleri yönetmek için
- **Bottom Tab Navigation:** uygulamanın ana modüllerine hızlı erişim sağlamak için

Bu yapı kullanıcıların uygulama içerisinde kolay ve hızlı bir şekilde gezinmesini sağlar.

3.3 Kullanıcı Arayüzü Yapısı

Mobil uygulamanın kullanıcı arayüzü kullanıcıların ürünleri kolay bir şekilde arayabileceği, ürünleri tarayabileceği ve analiz sonuçlarını görüntüleyebileceği şekilde tasarlanacaktır.

Uygulama içerisinde genel olarak şu işlemler gerçekleştirilebilecektir:

- ürün arama
- barkod ile ürün tarama
- kamera ile ürün adı tanıma
- ürün içerik analiz sonuçlarını görüntüleme
- favori ürünleri yönetme
- kullanıcı profil bilgilerini düzenleme

Kullanıcı arayüzü sade ve kullanıcı dostu olacak şekilde tasarlanacaktır.

3.4 Kamera ve Tarama Modülü

Uygulamada ürün tanıma işlemleri için kamera tabanlı bir tarama sistemi kullanılacaktır. Kamera işlemleri için **react-native-vision-camera** kütüphanesi kullanılacaktır.

Bu kütüphane yüksek performanslı kamera erişimi sağlayarak aşağıdaki işlemlerin gerçekleştirilmesini mümkün kılar:

- barkod tarama
- ürün adı tanıma (OCR)
- görüntü yakalama

Tarama işlemi sonucunda elde edilen veriler backend sistemine gönderilerek ürün eşleştirme işlemi yapılacaktır.

3.5 Ürün Analiz Arayüzü

Ürün analiz ekranı kullanıcıya ürünün içerik analiz sonuçlarını sunar. Bu ekranda ürünün içerik listesi ve içeriklerin güvenlik durumları görsel olarak gösterilecektir.

Analiz sonuçları kullanıcıya daha anlaşılır olması için renk kodları ile gösterilecektir. Bu yaklaşım kullanıcıların ürün içeriklerini hızlı şekilde değerlendirmesine yardımcı olur.

3.6 Backend Entegrasyonu

Mobil uygulama backend sistemi ile **REST API** üzerinden iletişim kuracaktır. Backend servisleri ürün verileri, ingredient analiz sonuçları ve kullanıcı bilgileri gibi verileri sağlayacaktır.

Mobil uygulama backend servislerine HTTP istekleri göndererek gerekli verileri alacak ve bu verileri kullanıcı arayüzünde gösterecektir.

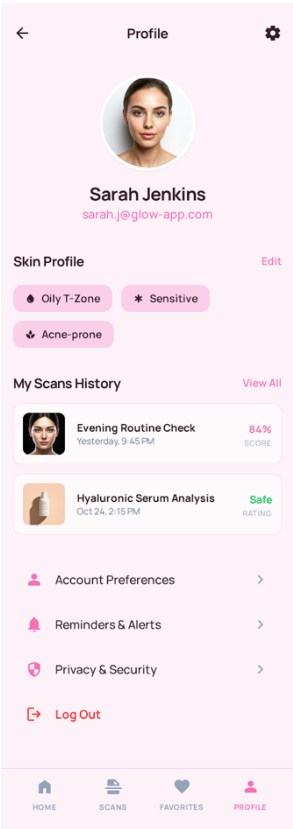
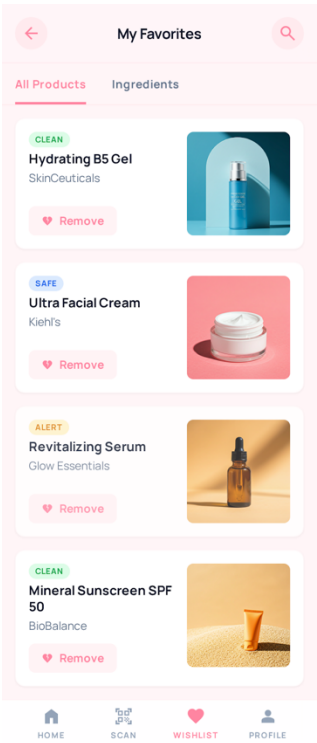
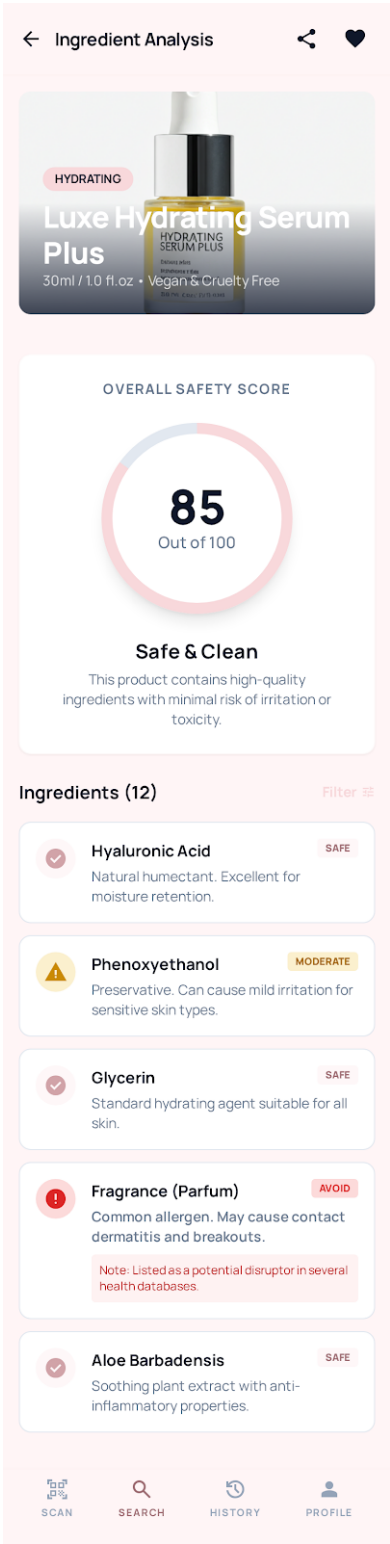
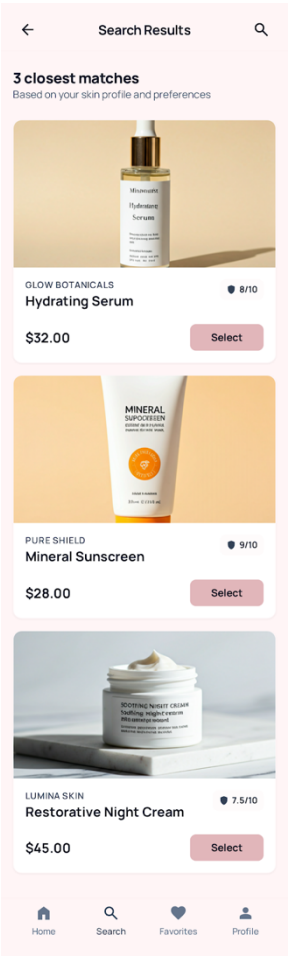
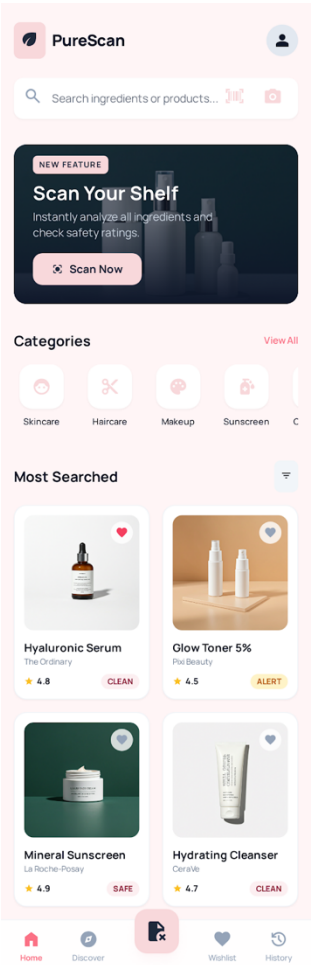
3.7 API İstekleri

Mobil uygulama içerisinde backend servisleri ile iletişim kurmak için **Axios** kütüphanesi kullanılacaktır. Axios, HTTP isteklerinin kolay bir şekilde gerçekleştirilmesini sağlayan bir JavaScript kütüphanesidir.

Backend API üzerinden gerçekleştirilecek temel işlemler şunlardır:

- ürün arama
- ürün detay bilgisi alma
- barkod ile ürün bulma
- ingredient analiz sonuçlarını alma
- kullanıcı işlemleri

UI Tasarımları:



4. Backend Tasarımı (.NET Web API)

4.1 Backend Teknolojisi

Backend sistemi **.NET 8 Web API** kullanılarak geliştirilecektir. .NET platformu yüksek performans, güçlü güvenlik özellikleri ve ölçeklenebilir mimari yapısı sayesinde modern web servisleri geliştirmek için uygun bir teknolojidir.

Backend sistemi aşağıdaki görevleri yerine getirecektir:

- mobil uygulama ile veri iletişimini sağlamak
- ürün verilerini yönetmek
- ingredient analiz işlemlerini gerçekleştirmek
- barkod ve OCR sonuçlarını işlemek
- kullanıcı işlemlerini yönetmek

Backend tüm servisleri REST API üzerinden sunacaktır.

4.2 Backend Proje Yapısı

Backend uygulaması katmanlı mimariye uygun şekilde geliştirilecektir. Bu yapı kodun modüler ve sürdürülebilir olmasını sağlar.

Önerilen proje klasör yapısı:

```
Backend
├── Controllers
├── Services
├── Repositories
├── Models
├── DTOs
├── Data
├── Middleware
├── Helpers
└── Configurations
```

Klasörlerin görevleri:

- **Controllers:** API endpointlerini yönetir
- **Services:** iş mantığını içerir
- **Repositories:** veritabanı erişim işlemlerini yapar
- **Models:** veri modellerini içerir
- **DTOs:** API veri transfer objeleri
- **Data:** veritabanı bağlantı işlemleri
- **Middleware:** özel request işleme katmanı

4.3 Controller Katmanı

Controller katmanı API endpointlerinin tanımlandığı katmandır. Mobil uygulamadan gelen HTTP istekleri bu katmanda karşılanır.

Backend içerisinde aşağıdaki controller yapıları bulunacaktır:

- AuthController
- ProductController
- IngredientController
- ScanController
- FavoritesController
- UserController

Controllerlar gelen istekleri ilgili servis katmanına yönlendirecektir.

4.4 Service Katmanı

Service katmanı uygulamanın iş mantığını içeren katmandır. Controller katmanından gelen istekler burada işlenir.

Backend içerisinde aşağıdaki servisler bulunacaktır:

- ProductService
- IngredientService
- ScanService
- SearchService
- UserService
- FavoriteService

Bu servisler uygulamanın tüm iş mantığını yönetir.

4.5 Repository Katmanı

Repository katmanı veritabanı işlemlerini yöneten katmandır. Bu katman veritabanı sorgularını ve veri erişimini yönetir.

Repository pattern kullanılması aşağıdaki avantajları sağlar:

- veritabanı işlemlerinin merkezi yönetimi
- kod tekrarının azaltılması
- test edilebilirliğin artması

Backend içerisinde aşağıdaki repository yapıları bulunacaktır:

- ProductRepository
- IngredientRepository
- UserRepository
- FavoriteRepository

4.6 Product Servisi

Product servisi ürünlerle ilgili tüm işlemleri yönetir.

Product servisi aşağıdaki işlemleri gerçekleştirir:

- ürün arama
- ürün detaylarını getirme
- barkod ile ürün bulma
- ürün ingredient listesini getirme

Örnek endpoint:

```
GET /products/search
```

Parametre:

```
?q=cleanser
```

4.7 Scan Servisi

Scan servisi barkod ve OCR işlemlerinden gelen verileri işler.

Scan servisi şu işlemleri gerçekleştirir:

- barkod verisini işleme
- ürün veritabanında barkod arama
- OCR metni ile ürün arama
- eşleşen ürünleri döndürme

Örnek endpoint:

```
POST /scan/ocr
```

Body:

```
{  
  "text": "cerave hydrating cleanser"  
}
```

4.8 Ingredient Analiz Servisi

Ingredient analiz servisi ürün içeriklerini analiz ederek güvenlik değerlendirmesi yapar.

Analiz süreci şu şekilde çalışır:

1. ürün ingredient listesi alınır
2. ingredient veritabanı ile eşleştirilir
3. her içerik için risk seviyesi belirlenir
4. ürün için genel güvenlik skoru hesaplanır

4.9 Hata Yönetimi (Error Handling)

Backend sisteminde hataların doğru şekilde yönetilmesi için merkezi bir hata yönetim sistemi kullanılacaktır.

Hata yönetimi için:

- global exception middleware
- standart hata mesaj formatı
- HTTP status code kullanımı

Örnek hata mesajı:

```
{
  "error": "Product not found",
  "status": 404
}
```

Bu yapı uygulamanın daha stabil çalışmasını sağlar.

4.10 Logging Sistemi

Backend sisteminde hataların ve sistem aktivitelerinin izlenebilmesi için bir logging sistemi kullanılacaktır.

Logging sistemi aşağıdaki bilgileri kayıt altına alacaktır:

- API istekleri
- sistem hataları
- performans metrikleri

.NET içerisinde logging için şu araçlar kullanılabilir:

- built-in .NET logging
- Serilog

Bu sistem uygulamanın bakım ve hata ayıklama süreçlerini kolaylaştırır.

5. Veritabanı Tasarımı

5.1 Veritabanı Teknolojisi

Uygulama için iki farklı veritabanı seçeneği değerlendirilecektir:

1. **PostgreSQL (Relational Database)**
2. **MongoDB (NoSQL Database)**

PostgreSQL ilişkisel veri yapıları için güçlü bir çözüm sunarken, MongoDB esnek veri modeli sayesinde hızlı geliştirme avantajı sağlar.

Projenin ilerleyen aşamalarında veri yapısına göre uygun veritabanı seçilecektir.

5.2 Veri Modeli

Uygulamada saklanacak temel veri türleri aşağıdaki gibidir:

- kullanıcı bilgileri
- kozmetik ürün bilgileri
- ingredient listeleri
- ingredient risk verileri
- favori ürünler
- tarama geçmişi

Bu veri türleri farklı tablolar veya koleksiyonlar halinde saklanacaktır.

5.3 Kullanıcı Tablosu (Users)

Kullanıcı bilgileri kullanıcı hesabı yönetimi için saklanacaktır.

PostgreSQL tablo yapısı:

```
users
      Alan      Tip
id      uuid
email    varchar
password_hash varchar
created_at timestamp
```

Bu tablo kullanıcı giriş işlemleri için kullanılacaktır.

5.4 Ürün Tablosu (Products)

Kozmetik ürün bilgileri bu tabloda saklanacaktır.

PostgreSQL tablo yapısı:

```
products
      Alan      Tip
id      uuid
name     varchar
brand    varchar
barcode  varchar
image_url text
created_at timestamp
```

Bu tablo ürün arama ve barkod eşleşmesi işlemleri için kullanılacaktır.

5.5 Ingredient Tablosu (Ingredients)

Kozmetik içerikleri bu tabloda saklanacaktır.

PostgreSQL tablo yapısı:

```
ingredients
  Alan      Tip
id          uuid
name        varchar
risk_level  varchar
description text
```

5.6 Ürün – Ingredient İlişkisi

Bir ürün birden fazla ingredient içerebilir. Bu nedenle **çoktan çoğa ilişki** kurulacaktır.

PostgreSQL tablo yapısı:

```
product_ingredients
  Alan      Tip
product_id  uuid
ingredient_id uuid
```

Bu tablo ürünlerin içerik listelerini saklamak için kullanılacaktır.

5.7 Favori Ürünler Tablosu

Kullanıcıların favori ürünleri bu tabloda saklanacaktır.

```
favorites
  Alan      Tip
id          uuid
user_id     uuid
product_id  uuid
created_at  timestamp
```

Bu tablo kullanıcıların favori ürünlerini listelemek için kullanılacaktır.

5.8 Tarama Geçmişi Tablosu

Kullanıcıların yaptığı ürün taramaları bu tabloda saklanacaktır.

```
scan_history
```

Alan	Tip
id	uuid
user_id	uuid
product_id	uuid
scanned_at	timestamp

Bu veri kullanıcı deneyimini geliştirmek amacıyla kullanılacaktır.

5.9 MongoDB Veri Yapısı

MongoDB kullanılması durumunda ürün verileri tek bir document içerisinde saklanabilir.

Örnek MongoDB document yapısı:

```
{
  name: "Hydrating Cleanser",
  brand: "CeraVe",
  barcode: "3337875597357",
  ingredients: [
    {
      name: "Glycerin",
      risk: "safe"
    },
    {
      name: "Parfum",
      risk: "moderate"
    }
  ]
}
```

} Bu yapı esnek veri modeli sağlar.

6. Computer Vision Sistemi (OCR ve Barkod Tanıma)

6.1 Computer Vision Modülünün Amacı

Uygulama kullanıcıların kozmetik ürünleri hızlı bir şekilde tanıyabilmesi için kamera tabanlı bir tarama sistemi kullanacaktır. Bu sistem ürünleri iki farklı yöntem ile tanıyacaktır:

1. barkod tarama
2. ürün adı tanıma (OCR)

Bu yöntemler sayesinde kullanıcılar ürünleri manuel arama yapmadan uygulamaya tanıtabilir.

6.2 Kamera Modülü

Mobil uygulamada kamera işlemleri için **React Native kamera kütüphanesi** kullanılacaktır.

Önerilen kütüphane:

- react-native-vision-camera

Bu kütüphane yüksek performanslı kamera işlemleri için geliştirilmiştir ve gerçek zamanlı görüntü işleme işlemlerini destekler.

Kamera modülü şu işlemleri gerçekleştirecektir:

- kamera görüntüsünü başlatma
- barkod algılama
- fotoğraf çekme
- OCR için görüntü yakalama

6.3 Barkod Tanıma Sistemi

Barkod tarama sistemi kozmetik ürünlerin barkod numarasını okuyarak veritabanında eşleşme yapar.

iOS platformunda barkod tanıma işlemi için şu framework kullanılacaktır:

AVFoundation

AVFoundation Apple tarafından geliştirilen bir medya işleme frameworküdür ve barkod tanıma işlemleri için yüksek doğruluk oranı sunar.

6.4 Barkod Tarama Süreci

Barkod tarama işlemi aşağıdaki adımlarla gerçekleşir:

1. kullanıcı tarama ekranını açar
2. kamera aktif hale gelir
3. kamera barkodu algılar
4. barkod numarası okunur
5. barkod numarası backend API'ye gönderilir
6. veritabanında barkod eşleşmesi yapılır
7. ürün bilgileri kullanıcıya gösterilir

Bu yöntem ürün tanıma için en hızlı yöntemdir.

6.5 OCR (Metin Tanıma) Sistemi

Barkod okunamadığı durumlarda ürün adı kamera ile tanınabilir. Bu işlem **OCR (Optical Character Recognition)**teknolojisi kullanılarak yapılacaktır.

iOS platformunda OCR işlemleri için aşağıdaki framework kullanılacaktır:

Apple Vision

Apple Vision frameworkü cihaz üzerinde çalışan bir bilgisayarlı görü sistemidir ve metin tanıma işlemlerinde yüksek doğruluk sağlar.

6.6 OCR İşleme Süreci

OCR sistemi aşağıdaki adımlarla çalışacaktır:

1. kullanıcı ürünün fotoğrafını çeker
2. görüntü Vision framework tarafından işlenir
3. görüntüden metin verisi çıkarılır
4. metin normalize edilir
5. metin backend API'ye gönderilir
6. ürün veritabanında arama yapılır
7. en yakın ürün eşleşmesi kullanıcıya gösterilir

Bu süreç barkod bulunamadığı durumlarda alternatif bir ürün tanıma yöntemi sağlar.

6.7 Metin Normalizasyonu

OCR işlemi sonucunda elde edilen metin doğrudan kullanılmadan önce temizlenmelidir.

Metin normalizasyonu şu adımları içerir:

- tüm harflerin küçük harfe dönüştürülmesi
- özel karakterlerin kaldırılması
- fazla boşlukların temizlenmesi
- gereksiz kelimelerin kaldırılması

Örnek:

OCR çıktısı:
CERAVE Hydrating Cleanser For Normal To Dry Skin

Normalize edilmiş metin:
cerave hydrating cleanser

Bu işlem arama algoritmasının doğruluğunu artırır.

6.8 Kamera Pipeline

Kamera işlemleri aşağıdaki pipeline üzerinden gerçekleşir:

```
Camera Frame
↓
Barcode Detection
↓
Barcode Found ?
↓ yes
Product Search
↓
Product Result
```

↓ no
Capture Image
↓
OCR Processing
↓
Text Extraction
↓
Search Algorithm
↓
Product Result

Bu pipeline ürün tanıma işleminin hızlı ve doğru şekilde gerçekleşmesini sağlar.

7. Ürün Arama Algoritması (Product Search Algorithm)

7.1 Arama Sisteminin Amacı

Ürün arama sistemi, kullanıcı tarafından girilen veya OCR ile elde edilen ürün adını veritabanındaki ürünlerle eşleştirerek doğru ürünü bulmayı amaçlar.

Bu sistem aşağıdaki durumlarda kullanılacaktır:

- manuel ürün arama
- OCR sonucu ürün adı tanıma
- alternatif ürün önerileri

Arama sistemi yüksek doğruluk ve hızlı sonuç üretme amacıyla tasarlanacaktır.

7.2 Arama Türleri

Uygulamada üç farklı arama yöntemi bulunacaktır:

1. **Manual Search**
Kullanıcı ürün adını yazarak arama yapar.
2. **Barcode Search**
Barkod numarası ile ürün bulunur.
3. **OCR Search**
Kamera ile tanınan ürün adı kullanılarak ürün bulunur.

Bu üç yöntem farklı algoritmalar kullanabilir ancak aynı ürün veritabanı üzerinde çalışacaktır.

7.3 OCR Arama Problemi

OCR ile elde edilen metinler çoğu zaman veritabanındaki ürün adıyla birebir eşleşmez.

Örnek:

Veritabanındaki ürün:

CeraVe Hydrating Facial Cleanser

OCR çıktısı:

cerave hydrating cleanser

Bu nedenle klasik string eşleşmesi yerine benzerlik algoritmaları kullanılacaktır.

7.4 Fuzzy Search Yaklaşımı

OCR aramalarında **fuzzy search** algoritması kullanılacaktır.

Fuzzy search iki metin arasındaki benzerliği hesaplayarak en yakın eşleşmeyi bulur.

Bu yöntem sayesinde küçük yazım hataları veya eksik kelimeler olsa bile doğru ürün bulunabilir.

7.5 Token Based Matching (Opsiyonel)

Bazı durumlarda kelimelerin sırası değişebilir. Bu nedenle token tabanlı eşleşme algoritması kullanılabilir.

Örnek:

hydrating cleanser cerave
cerave hydrating cleanser

Bu iki metin aynı kelimeleri içerdiği için yüksek benzerlik skoruna sahip olacaktır.

Token matching yöntemi OCR doğruluğunu artırır.

7.6 Arama Pipeline

OCR ile ürün arama süreci aşağıdaki pipeline üzerinden gerçekleşir:

```
OCR Text
↓
Text Normalization
↓
Candidate Product List
↓
Similarity Calculation
↓
Best Match Selection
↓
Product Result
```

Bu pipeline sistemin doğru ürün eşleşmesi yapmasını sağlar.

7.7 Benzerlik Skoru (Similarity Score)

Her ürün için bir benzerlik skoru hesaplanacaktır.

Örnek:

cerave hydrating cleanser

Sonuç:

CeraVe Hydrating Facial Cleanser → 92%

CeraVe Moisturizing Cleanser → 85%

Hydrating Face Wash → 60%

En yüksek skora sahip ürün kullanıcıya gösterilecektir.

8. Ingredient Analysis Engine

8.1 Analiz Sisteminin Amacı

Ingredient Analysis Engine, kozmetik ürünlerde bulunan içerikleri analiz ederek kullanıcıya içeriklerin güvenlik durumunu ve potansiyel cilt etkilerini göstermek amacıyla geliştirilmiştir.

Bu sistem ürün içeriğinde bulunan maddeleri analiz ederek aşağıdaki bilgileri üretir:

- içerik güvenlik seviyesi
- komedojenite (gözenek tıkama potansiyeli)
- içerik kategorisi
- ürün için genel güvenlik değerlendirmesi

Analiz sistemi backend tarafında çalışır ve mobil uygulamaya sonuçları API üzerinden gönderir.

8.2 Ingredient Veri Yapısı

Ingredient analiz sistemi her içerik için bir veri kaydı kullanacaktır.

Her ingredient için aşağıdaki bilgiler tutulacaktır:

- ingredient adı
- güvenlik etiketi
- komedojenite değeri
- açıklama

- kategori

Örnek veri yapısı:

```
Ingredient Name: Glycerin
Safety Label: Güvenli
Comedogenic Score: 0
Category: Humectant
Description: Cildi nemlendiren bir içeriktir.
```

Bu veri yapısı ingredient analizinin temelini oluşturur.

8.3 Ingredient Eşleştirme Süreci

Bir ürün analiz edildiğinde içerik listesi alınır ve ingredient veritabanı ile eşleştirilir.

Analiz süreci şu adımlarla gerçekleşir:

1. ürün ingredient listesi alınır
2. her içerik ingredient veritabanında aranır
3. ingredient kaydı bulunur
4. güvenlik etiketi alınır
5. komedojenite skoru alınır
6. analiz sonucuna eklenir

Bu işlem tüm ingredient listesi için gerçekleştirilir.

8.4 Analiz Pipeline

Ingredient analiz süreci aşağıdaki pipeline üzerinden çalışır:

```
Product Selected
↓
Ingredient List Retrieved
↓
Ingredient Database Matching
↓
Safety Label Extraction
↓
Comedogenic Score Extraction
↓
Product Safety Evaluation
↓
Analysis Result Returned
```

Bu pipeline sistemin düzenli ve hızlı şekilde analiz yapmasını sağlar.

8.5 Ürün Güvenlik Değerlendirmesi

Ürünün genel güvenlik durumu içeriklerin güvenlik etiketlerine göre değerlendirilir.

Örnek değerlendirme yaklaşımı:

- Çoğunluk **Tamamen Güvenli** ise ürün güvenli kabul edilir
- **Kabul Edilebilir** içerikler varsa kullanıcıya bilgi verilir
- Yüksek komedojenite içeren içerikler uyarı olarak gösterilir

Bu yaklaşım kullanıcıya basit ama etkili bir analiz sunar.

8.9 Komedojenite Uyarı Sistemi

Komedojenite skoru yüksek olan içerikler kullanıcıya özel olarak gösterilecektir.

Örnek:

Uyarı:

Bu ürün yüksek komedojenite değerine sahip içerikler içermektedir.

Bu özellik özellikle akneye yatkın kullanıcılar için faydalıdır.